

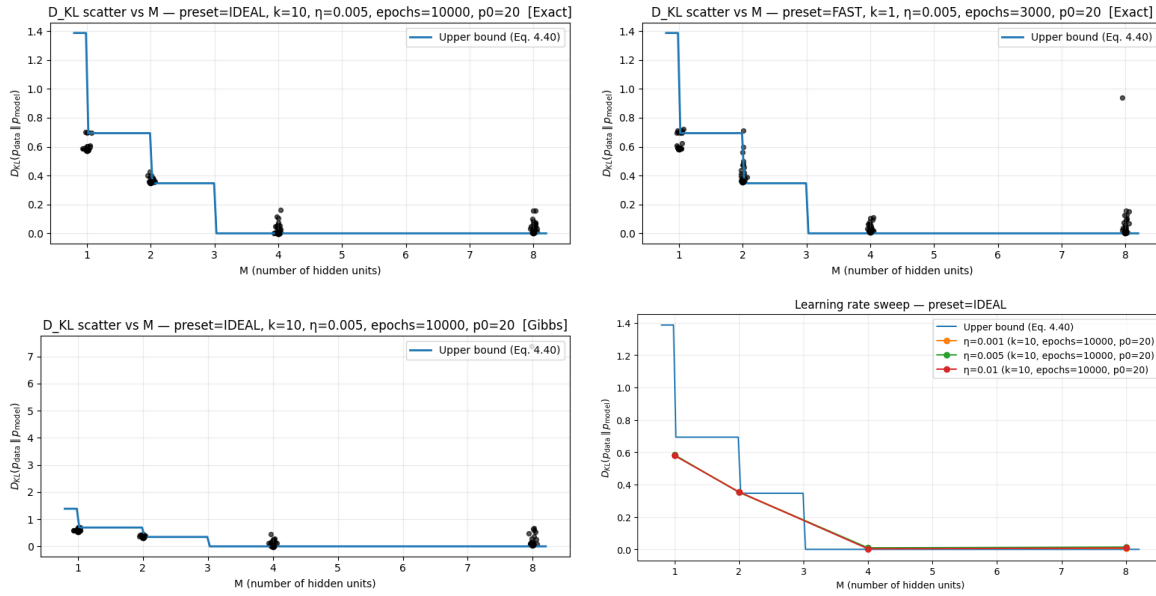


Figure 1: Kullback–Leibler divergence (D_{KL}) as a function of the number of hidden units M . The blue line shows the theoretical upper bound from Eq. (4.40). Top left: Fast preset ($k=1$, $\eta=0.005$, epochs=3000) with exact sampling. Top right: Ideal preset ($k=10$, $\eta=0.005$, epochs=10000) with exact sampling. Bottom left: Ideal preset with Gibbs sampling. Bottom right: Ideal preset with different learning rates $\eta \in \{0.001, 0.005, 0.01\}$. Each dot corresponds to an independent run (20 per M).

Method and Algorithm: I used two presets to test the algorithm, named *fast* and *ideal*. The “ideal” values were derived from Exercise 4.7 of the course book, while the “fast” ones were created to perform quicker initial tests and verify the correctness of the code. I used Algorithm 3 from the book, applying different values of k and different learning rates η . I also compared the Fast and Ideal presets between each other.

Results: The blue line in the figures is the theoretical upper bound, obtained from Eq. 4.40, representing the maximum theoretical value of D_{KL} . The experimental values follow this bound closely. The difference between exact and Gibbs sampling is more noticeable for the first two hidden layer sizes $M = 1, 2$; with few hidden units, the Gibbs chain does not have enough time to explore the energy landscape effectively. Regarding the learning rate η : with $\eta = 0.001$, updates are small and convergence is very slow but stable; with $\eta = 0.01$, updates are too large, causing noisy oscillations and occasional divergence; How long do I run the dynamics? The burn-in is with 10.000 steps and I used 100.000 samples. This setup yields stable probability estimates for Gibbs sampling. The outliers observed in $M = 2$ and $M = 8$ indicates reduced training stability in higher-dimensional parameter spaces, where stochastic noise and finite epochs prevent complete convergence.

Conclusions: The experiment confirms that increasing hidden units M reduces D_{KL} and improves representational capacity. A larger k (Ideal preset) yields a better approximation of the target distribution. A moderate learning rate ($\eta = 0.005$) offers the best balance between convergence speed and stability. The measured divergences follow the theoretical upper bound (Eq. 4.40), validating the RBM’s expected behavior.

Part of the code structure was organized with the assistance of OpenAI’s ChatGPT (GPT-5), also used for troubleshooting implementation errors.